

long long mat = 100000000 + 1;
long long L = a * b / gcd(a, b);

LL low = min(a, b)

LL high = n * min(a, b)

while (low <= high) {

long long mid = low + (high - low) / 2;

if ((mid / a) + (mid / b) - (mid / L) >= n) {

ans = mid;

high = mid - 1;

} else {

low = mid + 1;

}

}

return ans + mat;

}

int main() {

int n, a, b;

cin >> n >> a >> b;

cout << func(n, a, b) << endl;

return 0;

}

Time complexity: $O(\log(\frac{n}{\min(a, b)}))$

Space complexity: $O(1)$

Q1 Given three integers a, n, b return
 n^{th} magical no. modulo 10^9+7 .
 A positive integer is magical
 if the no. is either divisible by
 either a or b.

Ans-

Algorithm -

1. Initialize $M = 10^9+7$
2. Calculate $L = \text{lcm}(a, b)$
3. $\text{low} = \min(a, b)$
 $\text{high} = \min(a, b) \times n$
4. Binary Search Loop:
 while $\text{low} \leq \text{high}$
 $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$
 $\text{count} = (\text{mid}/a) + (\text{mid}/b) - (\text{mid}/L)$
 if $\text{count} < n$:
 $\text{low} = \text{mid} + 1$
 else if $\text{count} > n$:
 $\text{ans} = \text{mid}$
 $\text{high} = \text{mid} - 1$
5. return $\text{ans} \% (10^9+7)$

CODE -

```
#include <iostream>
#include <algorithm>
using namespace std;

int func (int n, int a, int b) {
```